



9

ANTI-DISASSEMBLY



Because disassemblers break down binary files into assembly code based on their own (often very complex) algorithms, there's some room for error. Malware authors are aware of this vulnerability and can actively exploit it. They may also attempt to obfuscate the malware's control flow or string and API function call references, making the code especially difficult to navigate statically. These are examples of *anti-disassembly* techniques, or ways in which malware complicates the process of reverse engineering code with a disassembler. In this chapter, we'll look at these tactics in depth and what malware analysts can do to address them.

Breaking Disassemblers

Disassemblers interpret a file based on their own hardcoded logic and assumptions, which means that they can interpret bytes in different and sometimes problematic ways. Code could be incorrectly disassembled into data, or vice versa, and bytes might be added to the wrong instructions, producing completely new and erroneous instructions.

As an example, the bytes `e8 8b 45 0a 90` can be disassembled into a call instruction. Removing the first byte (`e8`) would result in a completely different disassembled instruction. In this common anti-disassembly approach, known as the *rogue byte* technique, rogue bytes are inserted into the malware to confuse the disassembly process. Consider, for example, the following code snippet:

```
--snip--
00402100      b8 00 00 00 00  mov eax, 0x00
00402105...   85 c0                test eax, eax
00402107...   74 01                jz  loc_402109 + 1

loc_402109:
00402109      e8 8b 45 0a 90  call 0x900a4590
--snip--
```

Here you can see several disassembled instructions in the right column, the bytes that make up those instructions in the middle column, and the address offset in the left column. These disassembled instructions don't make much sense. For example, there's a jump-if-zero (`jz`) instruction with a target of `loc_402109 + 1`. This jump will always occur because the `mov` instruction prior to the `jz` instruction sets `eax` to 0, but the code jumps to the *second* byte of the next instruction (byte `8b`). The code also includes a call instruction to an address that doesn't even exist in this executable, since our executable is in the `0x00402xxx` address range, not the `0x900xxxxx` range. Let's take a closer look.

As [Chapter 3](#) explained, a disassembler doesn't always know how to differentiate code from data. This means that when it converts bytes to code, that code may in reality be data, or vice versa. The bytes that make up the `call 0x900a4590` instruction are `e8 8b 45 0a 90`. The first byte, `e8`, represents the call instruction in the x86 assembly instruction set. If we take out this byte, we're left with `8b 45 0a 90`. This series of bytes in x86 assembly is equivalent to the following code:

```
mov eax, [ebp+10]
nop
```

Here we have a `mov` instruction (to move the value stored on the stack at `ebp+10` to `eax`), followed by a `nop` instruction. This code makes a lot more sense than our original call instruction (`call 0x900a4590`). Thus, it seems that the first byte (`e8`) is a rogue byte, added to the code simply to confuse disassemblers.

You can deal with this by overriding incorrect code or data. In IDA, you can hit the C and D keys (C for converting data to code and D for converting code to data). In Ghidra, it's the opposite, confusingly enough; press C for converting code to data (C stands for “clear code bytes,” in this case) and D for converting data to code (D stands for “disassemble”).

If you select the bogus call instruction in IDA and press D, the instruction is broken into data, as shown here:

```
--snip--
00402100 mov eax, 0x00
00402105 test eax, eax
00402107 jz loc_402109 + 1

00402109 loc_402109:
❶ 00402109 db E8h
0040210A db 8Bh
0040210B db 45h
0040210C db 0Ah
0040210D db 90h
--snip--
```

Notice that what once was code is now data bytes, starting at ❶. Now, if you select the byte values starting at offset `0040210A` (taking care not

to select the `e8` byte) and continuing until `0040210D`, then press `C` to convert this to code, you get the following:

```
--snip--  
00402100 mov eax, 0x00  
00402105 test eax, eax  
00402107 jz loc_402109 + 1  
  
00402109 loc_402109:  
00402109 db E8h  
0040210A mov eax, [ebp+10]  
0040210D nop  
--snip--
```

The malware moves `0x00` into `eax` (in order to zero-out `eax`) and then uses a condition jump (`jz`); as noted earlier, the code will always take this jump. However, now the code jumps right over the rogue byte (`e8`) and executes the `mov` and `nop` instructions instead. This malware sample cleverly inserted the rogue byte in order to trick the disassembler into thinking that it was part of the original `call` instruction!

This is a fairly simple example of an anti-disassembly method, but it's a common one. This presents a challenge for both disassemblers and reverse engineers. When you encounter situations like this, in which code is simply incorrect or doesn't make sense, try manually converting the code to data bytes or some of the bytes into code. It may help you fix up the code so that you can better understand it.

Control Flow Obfuscation

The next anti-disassembly method we'll look at is *control flow obfuscation*, or adding unnecessary complexity to the malware code, making it much more difficult to analyze statically. This type of obfuscation can also flummox disassemblers, which may fail to properly disassemble the code.

To add this type of obfuscation, malware authors use specialized code obfuscators designed specifically for this purpose or malware packers, which we'll discuss in detail in [Chapter 17](#). Let's dig into some of the common methods used to obfuscate control flow. At the end of this section, we'll discuss a few general strategies to deal with these tactics.

Unnecessary Jumps

Malware authors may add unnecessary jump statements to break up the malware's code into smaller blocks (see [Figure 9-1](#)).

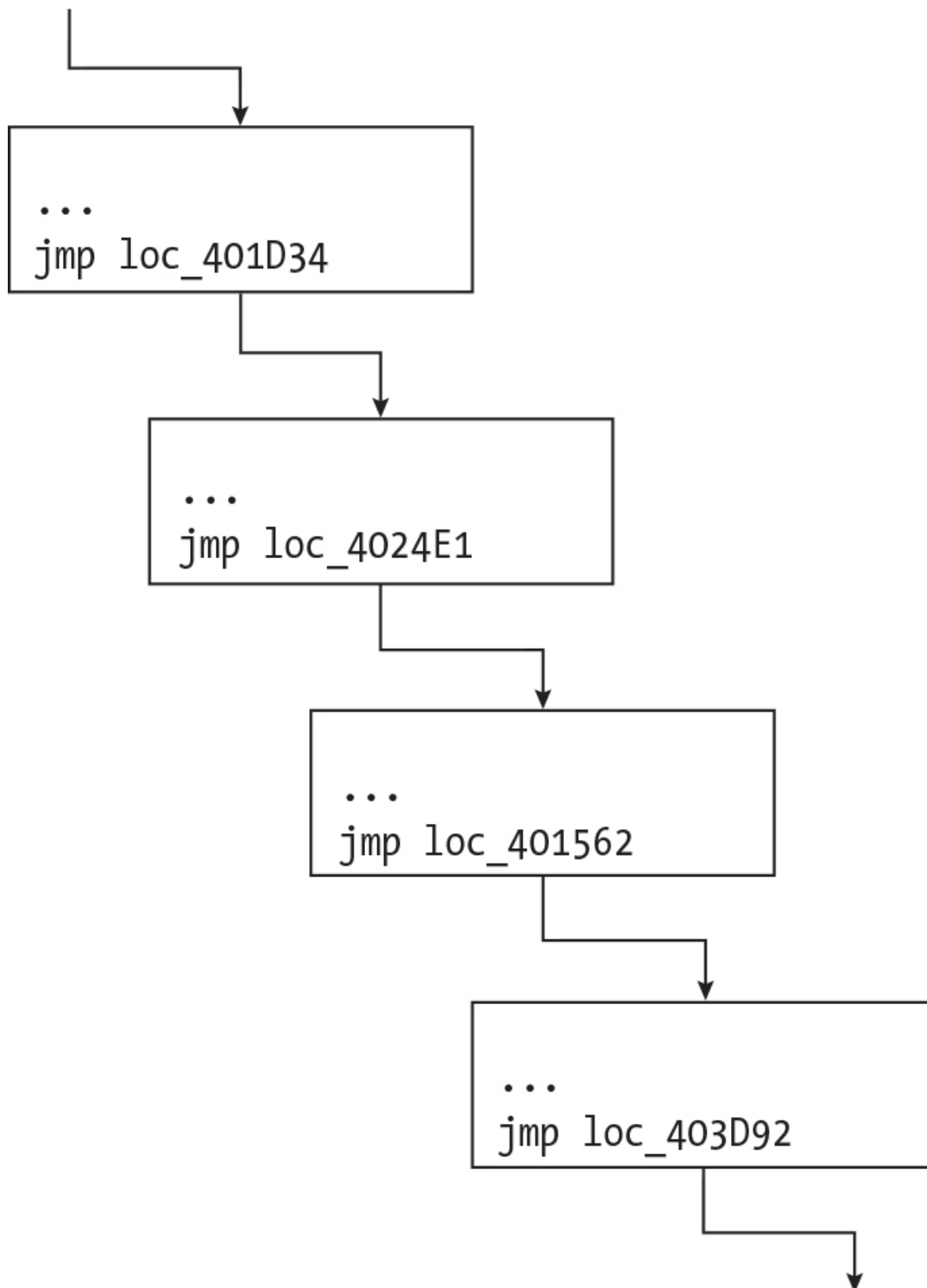


Figure 9-1: Unnecessary jump instructions

The code in **Figure 9-1** was once a single block, but an obfuscator has broken it into chunks, with each block connecting to the next with a jump statement. Functionally, the code is the same, but now reverse engineers will have more difficulty understanding and following it. This example is quite basic, but obfuscators can add a nearly infinite amount of complexity to code, as you'll see in the next few sections.

Obfuscators can also make code jump forward and backward frequently in order to make it harder to follow sequentially, as in this example:

```
--snip--
push 300h
jmp loc_402B20
--snip--
loc_402A30:
call Sleep
jmp loc_402B65
--snip--
loc_402B20:
pop ebx
jmp loc_402A30
--snip--
loc_402B65:
push ecx
--snip--
```

This code jumps around to different areas simply for the sake of confusion. It first jumps to `loc_402B20`, then back up to `loc_402A30`, and then back down to `loc_402B65`, creating a hard-to-follow code flow logic.

Unnecessary Code

Malware authors can add other types of unnecessary code to their malware. For example, they might create copies of code blocks or functions that are effectively the same, or at least very similar, so that the code can then be executed interchangeably, leading to the same final block of code, as shown in [Figure 9-2](#).

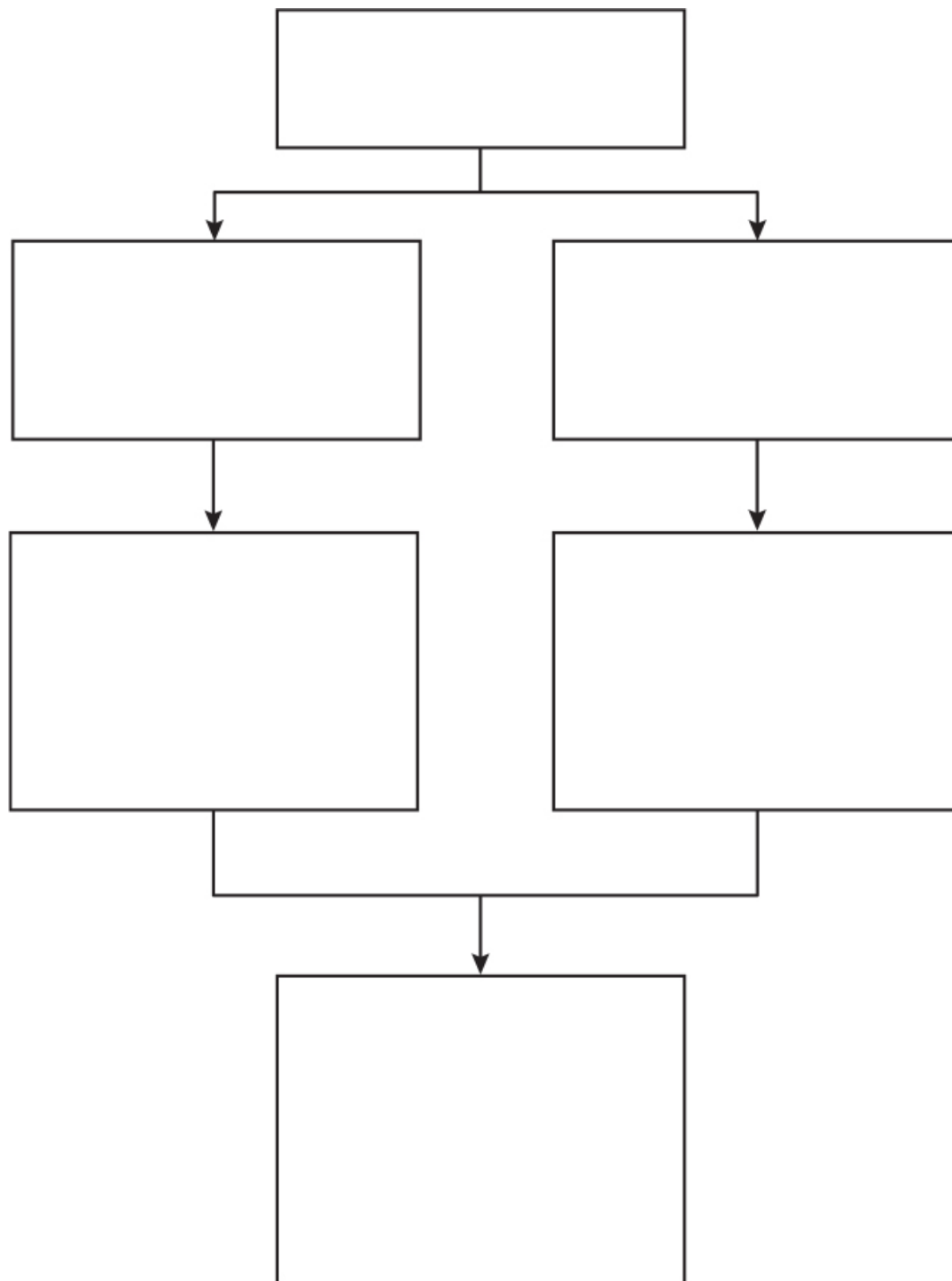


Figure 9-2: Interchangeable code blocks

This doesn't affect the malware's behavior but creates complexity for reverse engineers.

Alternatively, malware authors and obfuscators can add dummy code that will never be executed and exists only to confuse analysts, waste CPU cycles, and slow down the analysis process. This code could be

anything, so it's difficult to provide concrete examples of what it might look like, but this snippet demonstrates the technique:

```
--snip--  
inc ecx  
push ecx  
dec ecx  
push ecx  
--snip--
```

This code is simply incrementing the `ecx` register by 1, pushing this value to the stack, decrementing it by 1, and then pushing that value to the stack. It quite obviously serves no valid purpose.

Control Flow Flattening

Control flow flattening is a method of obfuscating control flow by compressing a sequence of conditional code blocks into a single block. This is usually accomplished via switch statements that direct control flow.

Figure 9-3 shows a program before control flattening has occurred.

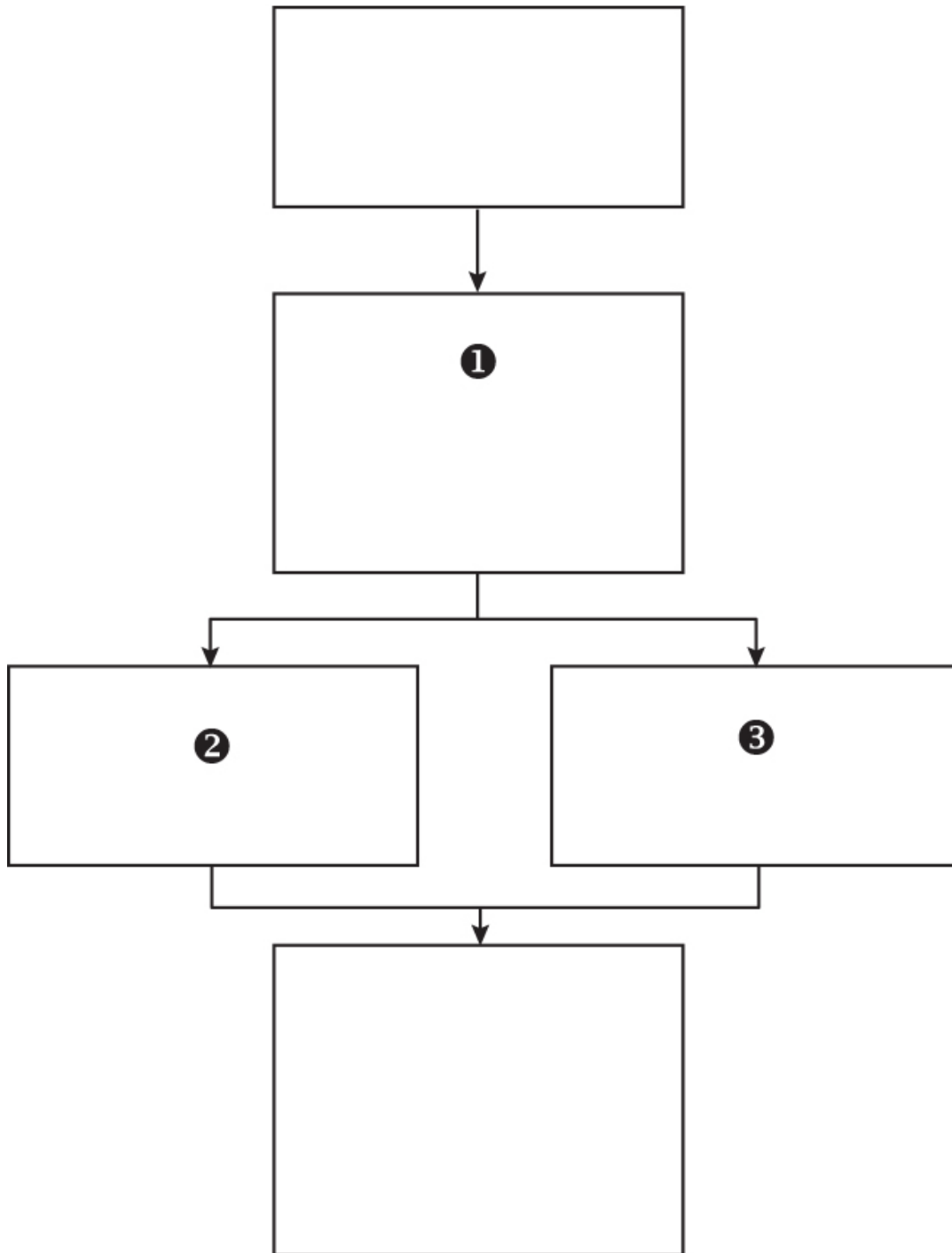


Figure 9-3: A program before control flow flattening is applied

This program represents normal, unobfuscated code. In the code block labeled ①, there's a conditional statement that will jump to one of two locations (code block ② or ③). If this program were run through a control flow-flattening algorithm, it might end up looking more like

Figure 9-4.

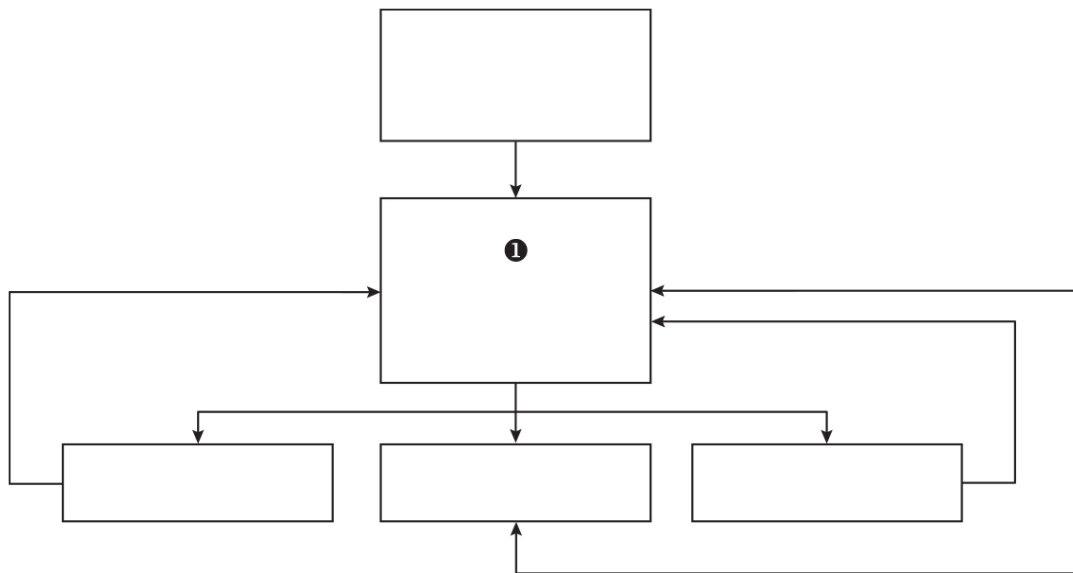


Figure 9-4: A program after control flow flattening is applied

In **Figure 9-3**, code block ❶ was responsible for the conditional statement that led to the jump to either code block ❷ or ❸. In the flattened code, a *central dispatch* code block ❶ is responsible for the conditional statement but also keeps track of where the code should “flow” next. After the dispatcher directs the control flow to a block of code, control is returned to the dispatcher, which directs the control flow further. The dispatcher adds complexity to the disassembled code, making it more difficult for an analyst to understand its purpose and where execution will flow next.

Opaque Predicates

An *opaque predicate* (see **Figure 9-5**) is a value that is known to the program’s author but not to the program or disassembler at runtime. The program’s creator (in our case, the malware author) knows that a certain expression will result in a specific value, for example, but neither we as reverse engineers nor our disassembler tools know this.

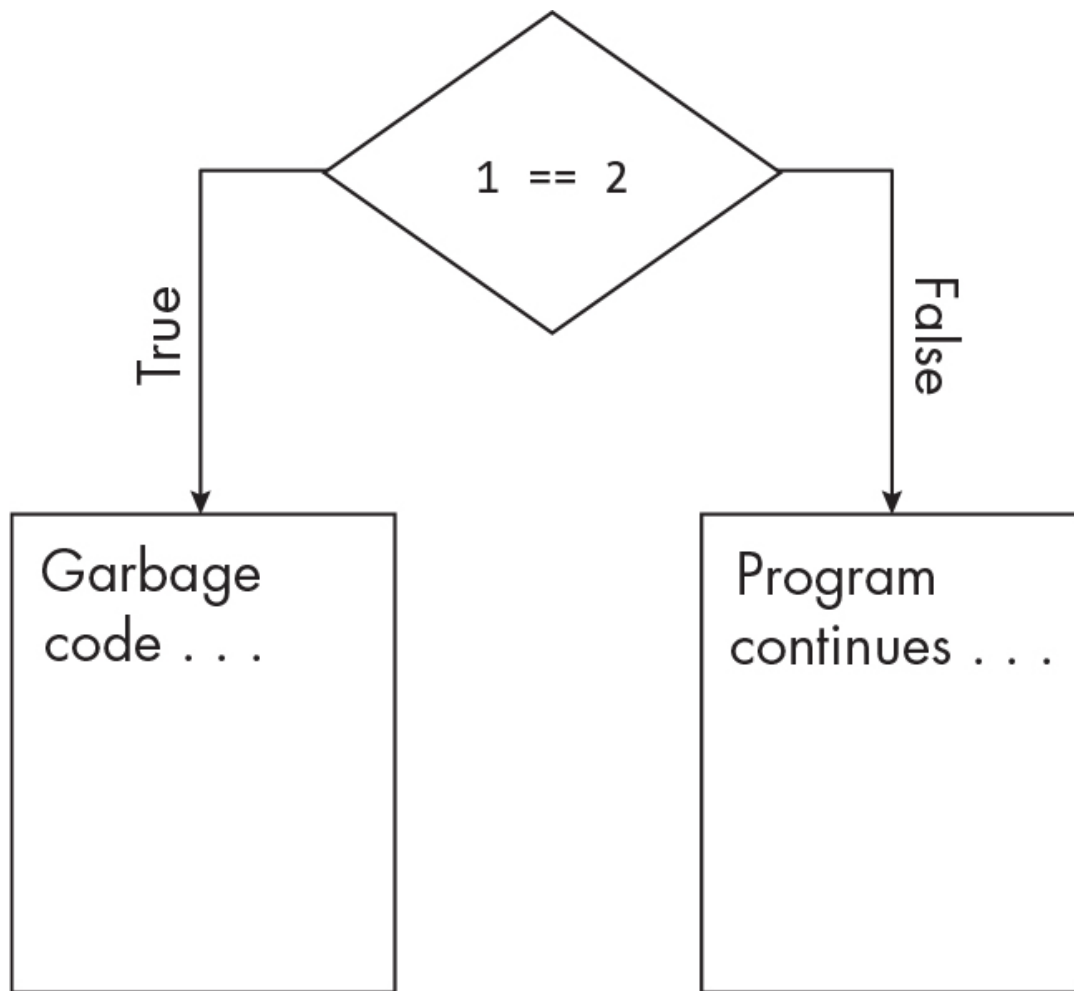


Figure 9-5: An opaque predicate in action

This code can take one of two paths, determined by the expression `1 == 2` (the opaque predicate). The malware author already knows that the program will take the branch to the right, but the analyst and the disassembler must manually analyze the logic to learn this. Obviously, this is a simplified example that almost anyone could decipher. However, malware authors can make an opaque predicate infinitely complex, for example, by calculating complicated mathematical functions at runtime.

This technique can also be combined with those previously mentioned, such as adding unnecessary code. The malware author could include a large amount of garbage code in the left branch that will never be executed. The reverse engineer must understand the opaque predicate before analyzing the rest of the program to avoid wasting time on the garbage code. Opaque predicates are difficult to deal with

and, as mentioned, can be as basic or complex as the malware author wishes. Often the best way to deal with them is to step through the malware in a debugger that will help reveal the true control flow.

Return Pointer Abuse

Another way to obfuscate control flow is with return (`ret`) instructions. For example, if a program executes Function B, once it reaches the end of that function, Function B will issue a `ret` instruction to return to its parent function (Function A). Before Function B can return, though, the program needs to know where to return to. Therefore, the return address is pushed to the stack before Function B executes and is popped off the stack once the `ret` instruction is executed. The following assembly code demonstrates this:

```
push returnAddress
--snip--
ret
```

This code issues a `push` instruction to push `returnAddress` and then executes the `ret` instruction, which will `pop` `returnAddress` off the stack to return the program's control flow to the parent function.

Malware can abuse the way return pointers work to replicate a `call` or `jmp` instruction. By pushing an address to the stack and then executing a `ret` instruction, the malware will force the control flow to execute the code at the new return address. This can confuse some disassemblers and generally makes following the code more difficult for the analyst as well.

SEH Handler Abuse

Malware can also take advantage of the *structured exception handler* (*SEH*), which stores a series of addresses for the pieces of code responsible for handling exceptions in Windows applications. When the ap-

plication raises an exception, its control flow transfers to one of the addresses stored in the SEH.

Malware can abuse the SEH by creating a new exception handler that points to malicious code. When the malware purposefully causes an exception in its code, the control flow will be transferred to code referenced in the exception handler. As a result, the analyst will need to know where the malware established the exception handler, as well as where the exception handler is pointing, in order to properly reverse engineer the code. Consider the following example:

```
--snip--  
mov eax, evil.429D8C  
push eax  
push dword ptr fs:[0]  
mov dword ptr fs:[0], esp  
--snip--
```

The focus of this code block is `fs:[0]`, which essentially points to the current exception handler. The malware replaces the default exception handler code with a pointer to malicious code (`evil.429D8C`). Once the malware triggers an exception, the code's control flow will be transferred to the address `evil.429D8C`. As there are no `jmp`, `ret`, or `call` instructions in use here, this control flow transfer can be difficult for the untrained eye to follow, so be on the lookout for code referencing `fs:[0]`. It's also common to see this followed by a `div` instruction, which might indicate that the malware is attempting to cause a division-by-zero exception. We'll discuss SEH and this specific code block further in [Chapter 11](#).

Function Pointer Abuse

As you've seen, a typical control flow transfer to a new function will involve a jump or call instruction. However, crafty malware can obscure these instructions by introducing function pointers like so:

```
--snip--  
mov [ebp+var_26], offset sub_4511D5  
call [ebp+var_26]  
--snip--
```

This malware sample moves the offset address of the function `sub_4511D5` into a variable on the stack, `var_26`. Then, it uses a call instruction and references the `var_26` variable, which contains the address of the target function it wishes to call (`sub_4511D5`).

This is a simple technique, but you can likely see how it might cause confusion during static analysis. To overcome this technique, you'd have to pinpoint the suspect call instruction and look backward through the code until you could identify what is stored in the referenced function pointer. Malware authors can make this obfuscation technique much more complex, however. For example, it can pass function offsets between different variables, which would make it very difficult for the analyst to identify the call's target function. Analyzing code such as this in a debugger can better help you understand what is going on.

Control Flow Obfuscation Countermeasures

This chapter has outlined only a few of the most common control flow obfuscation techniques, but you can overcome most of them with a few methods. First, you can use the same approach described in “Breaking Disassemblers” on [page 152](#). If you spot code that is impossible or simply doesn't make sense, try converting it into data. This may help you spot anomalies such as rogue bytes. The inverse is also true: if you spot data abnormalities or large sections of data in between code, try converting the data into code and reassessing it. This small tip may help you get around many simple anti-disassembly techniques.

Second, stepping through the code in a debugger can make a world of difference; it usually makes understanding the code and control flow much easier. The debugger can be used alongside the disassembler, and you can set a debugger breakpoint on the addresses of code that you don't entirely understand. If you spot a rogue byte in the code, for example, the debugger can help you understand what may be occurring. Some malware analysts like to use a disassembler with a built-in debugger (such as IDA Pro) for this very reason, but a separate disassembler and debugger will do just fine. I typically pair x64dbg with Ghidra or IDA.

Third, you can try to identify the obfuscator that was used on the malware. For example, tools such as Detect It Easy (DIE) and Exeinfo PE will attempt to identify possible obfuscators and packers (covered in [Chapter 17](#)). Once you've identified the obfuscator or packer, doing a bit of research on how it works may give you some insight into how you can reverse it, or there may even be a public deobfuscator available! Some tools attempt to generically deobfuscate code and remove some of the complexity, but in my experience they tend to not work very well and can leave holes in the code or misinterpret it. Finally, different disassemblers tend to disassemble code a bit differently. If you primarily use IDA, for example, give Ghidra or another disassembler a try and see if you get a result that's easier to understand.

Ultimately, dealing with anti-disassembly requires knowledge and experience of the assembly language, and there's no substitute for that. Learning assembly (x86, x64, or for whatever type of malware you're reversing) and continuing to build that skill set will help you more quickly identify the anti-disassembly and code obfuscation techniques being employed by malware.

API Call and String Obfuscation

In this section, you'll learn how malware can obfuscate its Windows API function calls and strings to hide its intentions from analysts.

NOTE

*This section outlines obfuscation techniques that are specifically applicable to anti-disassembly and protection against static analysis, but **Chapter 16** covers more generic obfuscation techniques. API call and string obfuscation can also be used for endpoint defense evasion, such as sidestepping anti-malware software, but **Part IV** will discuss this topic in more depth.*

Dynamic API Function Resolution

Dynamic API function resolution is when a program dynamically obtains the address of a function it wishes to call, rather than including the function in its import address table (IAT). The Windows API function `GetProcAddress` can assist with this. `GetProcAddress` retrieves the procedural address of a function inside a given module, and it takes two parameters: a handle to the module where the target function resides, and the name of the target function itself. Sometimes `GetProcAddress` is preceded by a call to `LoadLibrary`, which will load the module that contains the target function. Let's take a look at this in practice:

```
--snip--
push ecx ; "kernel32.dll"
call LoadLibraryA
push eax
push edx ; "IsDebuggerPresent"
call GetProcAddress
call eax
--snip--
```

This malware sample first pushes the name of the module that contains the target function (in this case, *kernel32.dll*) to the stack and invokes `LoadLibraryA`, which loads this library into the address space of the process. `LoadLibraryA` returns a handle to the *kernel32.dll* module, which is stored in `eax` and is then pushed to the stack (`push eax`). Next, the code pushes the name of the target function `IsDebuggerPresent` to the

stack and calls `GetProcAddress`. The call to `GetProcAddress` returns the address of the target function and stores it in `eax`. Finally, the malware executes a call instruction with the target of `eax`, which will subsequently invoke `IsDebuggerPresent`. As you can see, this technique adds a layer of obfuscation to the function call.

Jump Tables and Indirect API Calls

API calls can be obfuscated with *jump tables*, data structures that map addresses of external libraries. Jump tables can serve as a method both to obfuscate control flow and to hamper static code analysis.

Figure 9-6 shows what a jump table might look like in action.

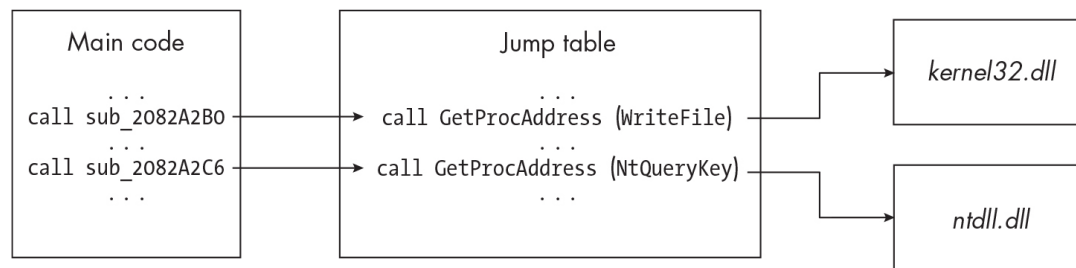


Figure 9-6: A jump table in action

In this simplified example, the malware's main code makes `call` instructions to different addresses representing the Windows API function the malware wishes to call. The malware's code then transfers control flow to the jump table, which in this case is essentially a list of further `call` instructions that use `GetProcAddress` to get the procedure address of the target Windows API function, and subsequently invokes that function. When the malware wishes to call `WriteFile`, for example, it makes a call to `sub_2082A2B0`, which jumps to the jump table, which in turn gets the address of `WriteFile` in the *kernel32.dll* library.

Jump tables can be as simple as a list of `call` instructions, as shown here:

```

sub_JumpTable:
    call sub_2052B2A0 ; jumps to code that further invokes WriteFile
  
```

```
call sub_2052B2B0 ; jumps to code that further invokes ReadFile
call sub_2052B2C0 ; jumps to code that further invokes IsDebuggerPresent
--snip--
```

API functions in the jump table can be dynamically resolved upon the malware's initial execution (thus building the table dynamically) or can be invoked by the malware as needed, meaning the function addresses are resolved on demand. This adds further complexity to the jump table, making the code more difficult for the reverser to follow.

Malware may also use indirect API calls. Similar to jump tables, API function addresses are dynamically resolved and stored in memory or in CPU registers for later use. Then, the malware invokes the function by issuing a call instruction for the address of the function rather than by function name. You can see this in the following 64-bit simplified code example:

```
--snip--
mov rcx, hModule ; "advapi32.dll"
mov rdx, "CryptEncrypt"
call GetProcAddress
mov [rbp-39], rax
mov rcx, hModule ; "advapi32.dll"
mov rdx, "CryptDecrypt"
call GetProcAddress
mov [rbp-35], rax
mov rcx, hModule ; "kernel32.dll"
mov rdx, "WriteFile"
call GetProcAddress
mov [rbp-31], rax
--snip--
```

This code is using indirect calls to obfuscate its function calls. First, it moves the name of the target function it wishes to call (`CryptEncrypt`) into `rdx`, as well as the associated module name (`hModule`), which, in the case of these functions, resides in *kernel32.dll*. Next, the code calls

GetProcAddress to get the address of the CryptEncrypt function. Then, the code moves this address onto the stack (`mov [rbp-39], rax`), which will be used later. The code runs this procedure twice more, for the functions CryptDecrypt and WriteFile. After storing the addresses of its target functions on the stack, the code can later invoke these functions by their addresses like so:

```
call [rbp-39]
```

This call instruction will invoke the function stored on the stack at `rbp-39`, which happens to be `CryptEncrypt`, a function used for encrypting data. Calling functions this way provides a layer of obfuscation for researchers who are manually reverse engineering the code.

Stack Strings

Stack strings refer to strings that are built on the stack dynamically in memory by the malware. They add a layer of obfuscation to a malware executable, making static analysis a bit more time-consuming, as shown here:

```
--snip--
mov    [ebp+file], 65h
mov    [ebp+file+1], 76h
mov    [ebp+file+2], 69h
mov    [ebp+file+3], 6Ch
mov    [ebp+file+4], 2Eh
mov    [ebp+file+5], 64h
mov    [ebp+file+6], 6Ch
mov    [ebp+file+7], 6Ch
lea    eax, [ebp+file]
push   eax
call   LoadLibraryA
--snip--
```

This code snippet contains several `mov` instructions, representing the code moving data onto the stack. What's interesting about them is that they're moving hex values, byte by byte, into a buffer (`ebp+file`). If you convert these hex values into ASCII (using the R key in IDA or selecting **Right-click ▶ Convert ▶ Char** in Ghidra), you can deobfuscate this stack string like so:

```
--snip--
mov    [ebp+file], 'e'
mov    [ebp+file+1], 'v'
mov    [ebp+file+2], 'i'
mov    [ebp+file+3], 'l'
mov    [ebp+file+4], '.'
mov    [ebp+file+5], 'd'
mov    [ebp+file+6], 'l'
mov    [ebp+file+7], 'l'
lea    eax, [ebp+file]
push   eax
call   LoadLibraryA
--snip--
```

Now you can make a more educated guess about what the malware is doing with this data. It's creating a string (`evil.dll`) on the stack and calling `LoadLibraryA`, which will load this malicious DLL file into the malware's process. This is a form of *process injection*, a technique **Part IV** will cover in depth.

There are some great tools malware analysts can use for automating stack string deobfuscation. Running a malware sample through FLOSS (discussed in **Chapter 2**), for example, can deobfuscate some basic string obfuscation and build an IDA script file so that you can easily load this data back into your IDA database. Here's an example of FLOSS output:

```
> FLOSS extracted 55 stackstrings
GetWindowsDirectoryA
```

```
VirtualAllocEx
GetSystemDirectoryA
Software\Microsoft\Windows NT\CurrentVersion\Windows
DeleteFileA
WriteFile
RegDeleteValueA
RegDeleteKeyA
ineIntel
GetUserNameA
CreateProcessA
recv
FindExecutableA
--snip--
```

Pestr (<https://pev.sourceforge.io>), another tool for stack string deobfuscation, can be run nearly the same way. Both tools are easy enough to quickly run before starting your reverse engineering process and may save you some time when analyzing malware code that has implemented basic string obfuscation.

Data Hashing

Malware authors can obfuscate malware functionalities by using data *hashing*, which is a kind of one-way data encoding; that is, it takes some data and encodes it into something else that can't be reversed. The ransomware family Maze uses the well-known *ROR-13* hashing algorithm to obfuscate Windows API function calls, as shown in the following code:

```
--snip--
mov [esp+38h+var_38], eax
mov [esp+38h+var_34], 7C0DFCAAh ; GetProcAddress
call sub_4011A0
sub esp, 8
mov [ebp+var_24], eax
mov eax, [ebp+var_4]
mov [esp+38h+var_38], eax
```

```
mov [esp+38h+var_34], 0EC0E4E8Eh ; LoadLibraryA  
call sub_4011A0  
--snip--
```

The `mov` instruction in the second line moves the ROR-13 hash `7C0DFCAAh`, the value of `GetProcAddress`, onto the stack. Similarly, the hash `0EC0E4E8Eh` represents the `LoadLibraryA` function, which moved to the stack in line 8. This malware is obfuscating its calls to the `GetProcAddress` and `LoadLibraryA` functions using hashes in place of the function name. There must be a function that is responsible for interpreting these hashes and loading the address of the target functions (in this case, function `sub_4011A0`), but this is not shown in the preceding code and the specifics of this are outside the scope of this chapter. However, this is well documented, such as in the blog post “Windows API Hashing in Malware” at <https://www.ired.team/offensive-security/defense-evasion/windows-api-hashing-in-malware>.

It’s difficult to understand what’s happening here simply by reviewing the code, since the function names are hashed and therefore unreadable. Luckily, many disassemblers have special features or plug-ins that can automatically identify potential hashed function names. In my case, the IDA plug-in *apihashes* was able to correctly identify and annotate the ROR-13–hashed data. Hashing will be discussed in greater detail in [Chapter 16](#).

Summary

In this chapter, you learned about several anti-disassembly techniques malware might employ to protect itself from malware analysts and their tools. Deobfuscating assembly code is a challenging task that requires a high level of technical skill and knowledge of a malware’s behavior and characteristics. Compounding this challenge is the fact that many of these techniques are very simple for malware authors to implement, thanks to special code compilers, obfuscators, and tools such as packers. It’s often much easier for malware authors to implement

anti-disassembly measures than it is for reverse engineers to circumvent them, but fighting back against such techniques is crucial to understanding the malware's behavior and functionality. As an analyst, you should use the range of tools and techniques at your disposal to deobfuscate malware code and reveal its true intentions.

In the next chapter, we'll discuss another anti-reversing technique that some malware implements to thwart dynamic code analysis: anti-debugging.